

Reverse Number of Swaps

Company: Amazon

Round: Online Assessment (OA)

Year: 2026

Difficulty: Easy

Topic: Arrays, Sorting Simulation, Selection Sort Variant

Source: [Amazon Online assessment](#)

Problem Description

Given an array of unique integers, a specific sorting algorithm sorts it using **selection-style swaps**. For every index i from left to right, the algorithm scans all elements to its right; **each time** it finds an element smaller than the current value at index i , it immediately swaps them (rather than just tracking the minimum and swapping once at the end of the scan). Return the total number of swaps performed to fully sort the array.

Input Format

- The first line contains a single integer N — the number of elements in the array.
- The second line contains N space-separated integers denoting the array arr .

Output Format

- Print a single integer — the total number of swaps performed.

Constraints

- $1 \leq N \leq 10^4$
- All elements of arr are unique integers.

Sample Input / Output

Example 1

Input:

4

5 4 1 2

Output:

5

Example 2

Input:

3

8 2 3

Output:

2

Explanation

Example 1: [5, 4, 1, 2]

- $i = 0$: compare with $j = 1$ ($4 < 5$) $\rightarrow$ swap $\rightarrow$ [4, 5, 1, 2]; compare with $j = 2$ ($1 < 4$) $\rightarrow$ swap $\rightarrow$ [1, 5, 4, 2]; compare with $j = 3$ ($2 < 1$? No) $\rightarrow$ no swap.
- $i = 1$: compare with $j = 2$ ($4 < 5$) $\rightarrow$ swap $\rightarrow$ [1, 4, 5, 2]; compare with $j = 3$ ($2 < 4$) $\rightarrow$ swap $\rightarrow$ [1, 2, 5, 4].
- $i = 2$: compare with $j = 3$ ($4 < 5$) $\rightarrow$ swap $\rightarrow$ [1, 2, 4, 5].
- Total swaps = **5**, matching the given trace exactly.

Example 2: [8, 2, 3]

- $i = 0$: compare with $j = 1$ ($2 < 8$) $\rightarrow$ swap $\rightarrow$ [2, 8, 3]; compare with $j = 2$ ($3 < 2$? No) $\rightarrow$ no swap.
- $i = 1$: compare with $j = 2$ ($3 < 8$) $\rightarrow$ swap $\rightarrow$ [2, 3, 8].
- Total swaps = **2**.

Approach to Solve This Problem:

This is a direct **simulation** of the described algorithm — it is *not* standard selection sort (which swaps at most once per outer iteration, using the minimum found). Here, a swap happens **every time** a smaller element is found during the inner scan, so a single outer index i can trigger multiple swaps.

1. **Outer loop (i from 0 to $N-1$):** treat $\text{arr}[i]$ as the "current pivot" for this position.
2. **Inner loop (j from $i+1$ to $N-1$):** compare $\text{arr}[j]$ with the **current** $\text{arr}[i]$ (which may have already changed due to an earlier swap in this same inner loop).
 - If $\text{arr}[j] < \text{arr}[i]$, swap them immediately and increment the swap counter.
 - Continue scanning with the *new* $\text{arr}[i]$ value.
3. After the inner loop finishes, $\text{arr}[i]$ is guaranteed to hold the smallest element among $\text{arr}[i..N-1]$, so the prefix $\text{arr}[0..i]$ is correctly sorted — this is what makes the process still converge to a fully sorted array, just with extra intermediate swaps compared to classic selection sort.
4. Return the total swap count.

Because this is a straightforward simulation with no shortcut needed for $N \leq 10^4$, brute-force nested loops ($O(N^2)$) comfortably fit within typical OA time limits.

Java Solution:

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        System.out.println(countSwaps(arr));
    }

    static int countSwaps(int[] arr) {
        int n = arr.length;
        int swaps = 0;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[i]) {
                    int temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                    swaps++;
                }
            }
        }
        return swaps;
    }
}
```

Solution:

1. **Read the input:** parse N and the array arr.
2. **Outer index i:** for each position, we want the correct (smallest remaining) value to end up there.
3. **Inner index j:** scan rightward from i + 1; whenever a smaller value than the current arr[i] is found, swap immediately and bump the counter — unlike classic selection sort, this may swap arr[i] more than once per outer iteration as smaller and smaller candidates are discovered.
4. **After each outer iteration:** arr[i] holds the minimum of the unprocessed suffix, so the array is sorted incrementally from left to right.
5. **Output:** the accumulated swaps counter once all positions are processed.

Complexity Analysis:

Time Complexity: $O(N^2)$

The algorithm uses two nested loops over the array — for each of the N outer positions, the inner loop scans up to N remaining elements, giving a worst-case quadratic number of comparisons (and potentially swaps).

Space Complexity: $O(N)$

The array is sorted in place using only a constant number of extra variables (i, j, temp, swaps); no auxiliary data structures are used.